



BCM5880X

SmartNIC Solution

User Guide

Broadcom, the pulse logo, Connecting everything, NetXtreme, Stingray, FlexSPARX, Avago Technologies, Avago, and the A logo are among the trademarks of Broadcom and/or its affiliates in the United States, certain other countries, and/or the EU.

Copyright © 2018-2020 Broadcom. All Rights Reserved.

The term “Broadcom” refers to Broadcom Inc. and/or its subsidiaries. For more information, please visit www.broadcom.com.

Broadcom reserves the right to make changes without further notice to any products or data herein to improve reliability, function, or design. Information furnished by Broadcom is believed to be accurate and reliable. However, Broadcom does not assume any liability arising out of the application or use of this information, nor the application or use of any product or circuit described herein, neither does it convey any license under its patent rights nor the rights of others.

Table of Contents

1 Overview	4
1.1 Purpose and Audience	4
1.2 References	4
1.3 SmartNIC Hardware Platform Overview	5
1.4 SmartNIC Software Components	6
1.5 SmartNIC Packet Flow	6
2 SmartNIC Pairing Models	7
2.1 SmartNIC Interface Pairing Model	7
2.2 SmartNIC Representor Pairing Model	8
2.3 Pairing Model Packet Flow	9
2.4 SmartNIC Software Infrastructure Implementation	10
2.5 User Space Configuration Commands	10
2.6 Geographical Numbering of Hosts, Physical Functions, and Virtual Functions	10
2.7 Create SmartNIC Representor Pairs	11
2.8 Create SmartNIC PF Pairs	12
2.9 Pair Delete	12
2.10 Pair Query	12
2.11 DPDK Representor Enhancement For Pairing	13
2.12 DPDK Datapath Support for SmartNIC Representors	13
2.13 Enable OVS Forwarding	14
2.14 Disable OVS Forwarding	14
2.15 Standard and Custom Tunnels	14
2.16 Bnxt-ctl Commands for Tunnels	15
2.17 Enabling and Disabling of Custom Tunnels	15
2.18 Configuring Tunnel Type Redirection	16
2.19 Custom Tunnel UPAR Overview and Constraints	16
2.20 In Service Software (Hot) Upgrade	17
2.21 ISSU Infrastructure Implementation	18
2.22 ISSU User Space Configuration Commands	18
2.23 User Space Configuration Commands Examples	19
Appendix A: Acronyms and Abbreviations	20
Revision History	22

1 Overview

1.1 Purpose and Audience

This document is focused on SmartNIC solutions using Broadcom NICs with the Stingray® BCM5880X system-on-chip (SoC) being the initial target. This document contains a description of software infrastructure developed for SmartNIC use cases.

The BCM5880X is a chip that integrates an enterprise-class Ethernet controller (Nitro) and a high-performance octal ARM Cortex-A72 core SoC. A primary set of use cases for BCM5880X is intended for SmartNIC. For SmartNIC applications, the BCM5880X appears and connects as a multifunction SR-IOV capable PCIe NIC endpoint to one or more host systems, typically server class x86 systems. Hosts may be based on Linux, FreeBSD, Windows, or VMWare and can make use of standard L2 kernel drivers to provide host Ethernet networking support. In addition, the host may use the standard DPDK poll mode driver to provide high-performance userspace-based Ethernet networking for network function virtualization (NFV) type applications.

For SmartNIC use cases, the OS running on the BCM5880X is Linux-based. The integrated Ethernet controller (Nitro) also appears and connects as a multifunction SR-IOV capable PCIe NIC endpoint to the SoC host. Standard L2 drivers provide kernel and userspace-based Ethernet Networking to the embedded SoC.

This document describes the software infrastructure that is required for supporting SmartNIC use cases. The main focus is the use of the CFA block of Nitro to steer the flow of packets through various classification and processing stages which may include software processing of all or selected packet flows on Stingray's embedded A72 cores.

1.2 References

The references in this section may be used in conjunction with this document.

For Broadcom documents, replace the "Xx" in the document number with the largest number available in the repository to ensure that you have the most current version of the document.

Document (or Item) Name	Number	Source
Broadcom Items		
<i>BCM5880X Hardware Design Guide</i>	5880X-DG1Xx	CSP
<i>BCM573XX NetXtreme® NVRAM Access</i>	5730X-AN2Xx	CSP
<i>BCM574XX NetXtreme® NVRAM Access</i>	5740X-AN2Xx	CSP
<i>PS225 Data Sheet</i>	PS225-HXX-DS1Xx	CSP
<i>5880X Data Sheet</i>	5880X-DS1Xx	CSP

1.3 SmartNIC Hardware Platform Overview

The BCM5880X is the main SmartNIC board used for development, testing, and production. There are three SKUs available with different amounts of onboard DRAM.

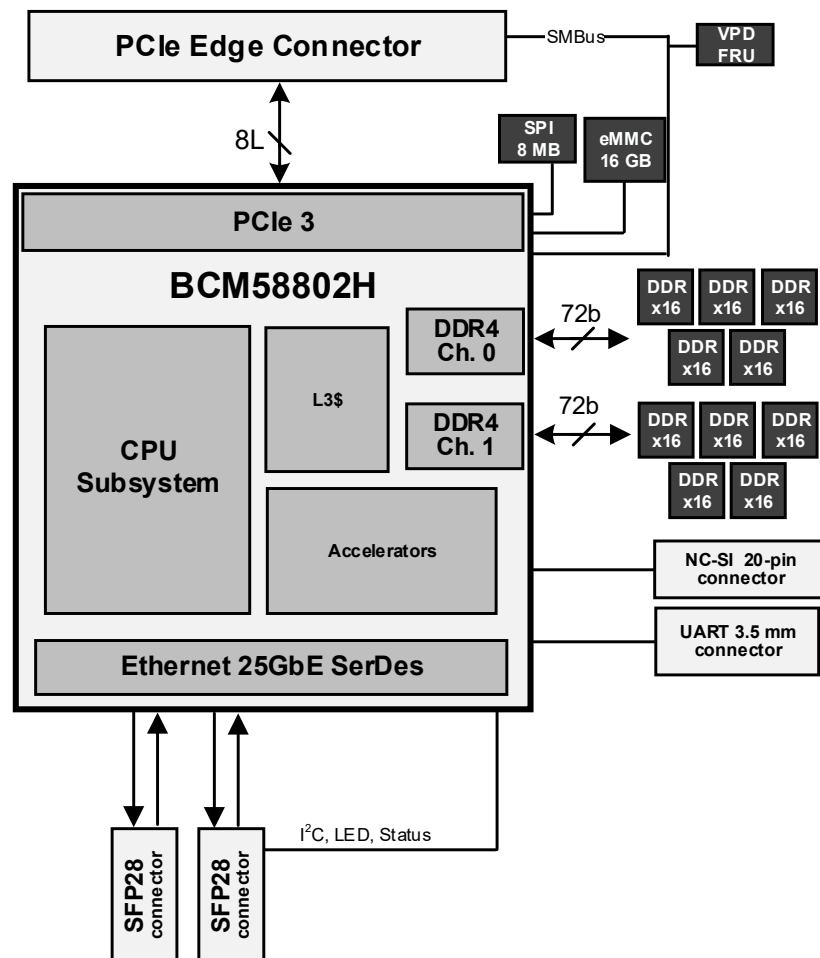
Table 1: BCM5880X Hardware Platforms

Board	Onboard DRAM Size	Ports
BCM958802A8046	16G	2 x 25G SFP ports
BCM958802A8048	8G	2 x 25G SFP ports
BCM958802A8044	4G	2 x 25G SFP ports

The UART capability for these cards is provided through UART0, which is the phono jack (3.5 mm) on the faceplate. This is initially the Nitro UART, the boot code switches this to be A72 UART. A special cable, phono jack to USB, is provided.

Figure 1 shows the main functional blocks of the BCM5880X-V2 SmartNIC adapter cards.

Figure 1: Functional Block Diagram



For additional information about the board, see the BCM5880X Data Sheet (PS225-HXX-DS1Xx), *Dual-Port 25 Gb/s Ethernet PCI Express SmartNIC Adapters*.

1.4 SmartNIC Software Components

Any discussion about the BCM5880X cannot be made without the knowledge of how the software is divided between the northbound and southbound sides of the chip.

The northbound side is not on the BCM5880X. Instead, northbound refers to the host that the BCM5880X is connected to via PCIe. The host is a server running Linux, FreeBSD, Windows, VMWare, and so forth, that sees the BCM5880X as a PCIe endpoint device.

The southbound side is on the BCM5880X. The southbound refers to the ARM SoC complex on the chip. Running on the southbound side is the Broadcom LDK with full support for the open source DPDK framework to enable easy development of data plane applications. Included within these frameworks is the BCM5880X specific APIs for accessing the firmware development kit and the blocks within the FlexSPARX™ 4 (for example, compression, encryption, and so forth).

The software sits in the middle, between the northbound and southbound sides is a PCIe bus and Nitro.

The primary software infrastructure components developed for SmartNIC in NXS 1.1 release are:

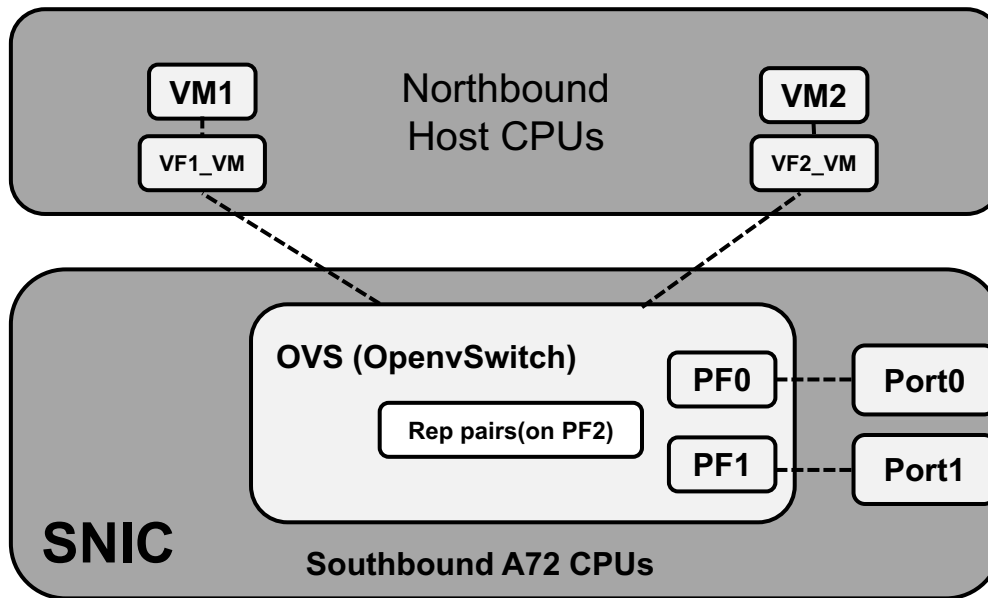
- Chimp firmware supports of Interface pairing and SmartNIC representor pairing, custom tunnels, and ISSU (in service software upgrade).
- User space utility for the user to manage pairs and tunnels (bnxt-ctl).
- DPDK enhancement (mostly in poll mode driver) for SmartNIC representor pairing.
- Firmware support of thermal management.
- CFA
- RoCE

The BCM5880X board comes out of manufacturing with a default 8 + 8 PF configuration. The default configuration does not support SR-IOV or pairing models. It is supplied as a common base config from manufacturing with the MAIA accessible using IP address 192.168.1.10. This configuration needs to be updated in order to access the SmartNIC features of this 2-port 25G NIC. The release supports a reference interface pairing configuration that allows up to 64 VF pairs and a reference SmartNIC representor pairing configurations that allows up to 128 representor pairs. Customers must use the provided tools to upgrade default images to the desired SmartNIC configuration.

1.5 SmartNIC Packet Flow

Figure 2 shows a typical SmartNIC packet flow using SmartNIC representor pairs and OVS software switching as the example application offloaded to southbound side. Virtual Machines running on northbound side host CPUs use VFs to transmit and receive packets same as the traditional SR-IOV case. SmartNIC representor pair connects the VF on northbound side to its representor on southbound side. All packets sent out by the VM pass through this high-performance point-to-point link and reach A72 CPUs for additional processing, which is an OVS software switching application in the following example. PF0_host, PF1_host, and VF1_VM, VF2_VM are functions exposed by Nitro to the northbound side. PF2_OVS and PF0_OVS are functions exposed by Nitro to the southbound side.

Figure 2: SmartNIC Packet Flow



The dotted lines show offloaded packets flowing between VM-VM and VM-network ports.

2 SmartNIC Pairing Models

The interface pairing enables virtual point-to-point Ethernet links to be created between an interface on the BCM5880X SoC and an interface on one of the x86 hosts.

The primary application enabled by interface pairing is software switching. An example of this would be a DPDK switching application executing on the SoC. The switching application would have physical Ethernet ports as well as several x86 host interfaces.

Any applications that benefit from high-performance point-to-point Ethernet connectivity can be enabled with interface pairing. For example, CPU processing of a storage software stack could be offloaded from northbound side (host CPUs) to southbound side (A72 CPUs) by passing traffic over an interface pair.

The current NXS release supports two interface pairing models, each functionally similar, but supporting different virtualization environments and scales.

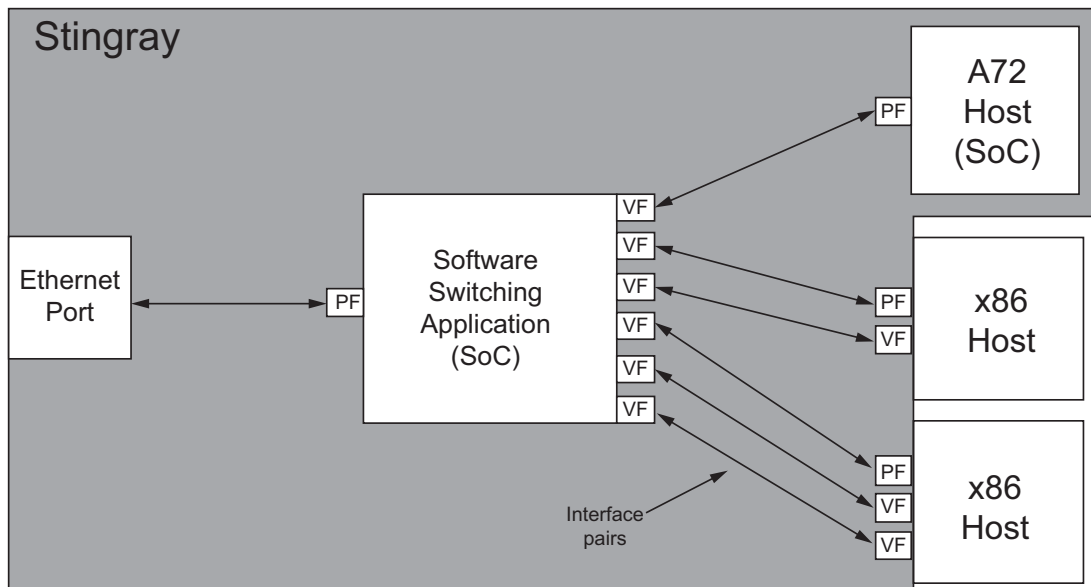
2.1 SmartNIC Interface Pairing Model

Typically, pairing is between a VF provisioned on the SoC and a VF provisioned on an x86 host. However, pairing is not limited to VFs. For example, a VF on the SoC may be paired with a PF on an x86 host. It is possible to pair a function on any host with a function on the same host, or on any other host. The SmartNIC Interface Pairing Model is used when pairing is between two functions. The BCM5880X supports 128 VFs divided among the x86 hosts and 64 VFs on the SoC. The 64 VFs on the SoC effectively limit the scale of the SmartNIC Interface Pairing Model to 64 pairs.

Since each end of the pair is a function (VF or PF), there is a lot of flexibility for how these functions may be used on both the SoC and on the x86 hosts. The kernel driver may be bound to functions to provide an interface into the kernel networking stack. Alternatively, functions may be driven by DPDK userspace poll mode drivers. Typically the switching application on the SoC is DPDK-based, but DPDK may also be executed in the x86 host or within a VM on the x86 to enable NFV type applications.

Figure 3 provides a functional diagram showing the SmartNIC Interface pairing model.

Figure 3: Use Case: Interface Pairing with the SmartNIC Interface Pairing Model



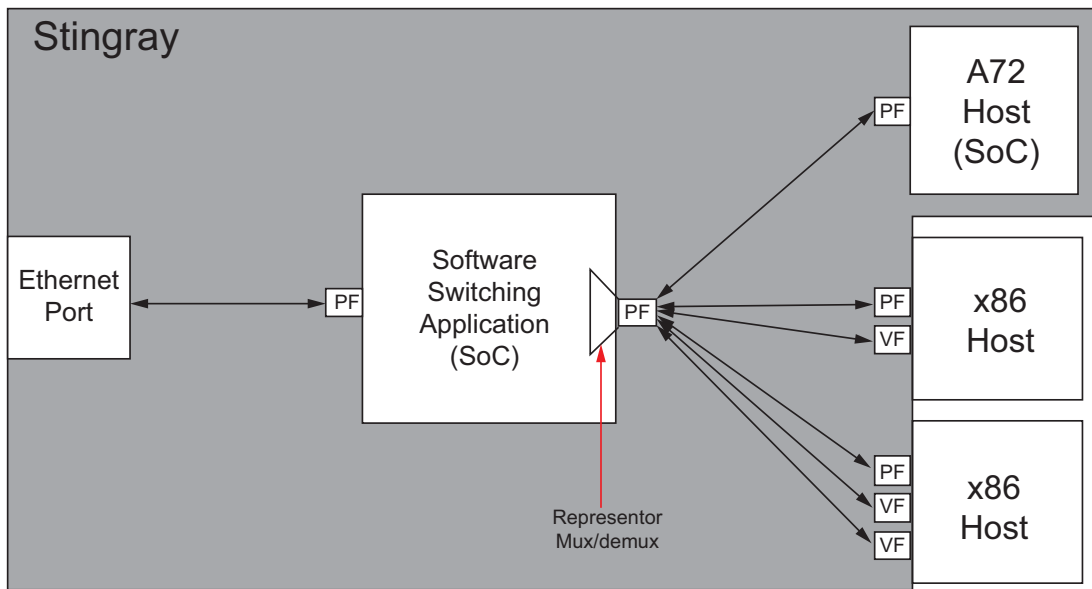
2.2 SmartNIC Represor Pairing Model

To extend the scale beyond 64 interface pairs (up to 128 pairs), the SmartNIC represor pairing model can be used. For software switching, a single SoC application may terminate a large scale of paired interfaces. The SmartNIC Represor Pairing Model enables a single PF on the SoC to demultiplex and multiplex multiple pairing interfaces which are paired with functions (VFs and/or PFs) on the x86 hosts.

The model enhances DPDK poll mode driver support for the switching application on the SoC. The poll mode driver will pass metadata received from Nitro with the packet in each DPDK packet buffer (mbuf), enabling the DPDK application to identify the receive interface of the packet. Likewise, the application can set metadata in the DPDK packet buffer prior to transmit. The poll mode driver will pass this metadata to Nitro with the transmit packet enabling the packet to be steered to the associated paired function.

Typically, each VF on the x86 host is passed to VMs to support a VM-based virtualization model. For VM-based virtualization, the maximum scale of VMs is determined by the number of CPU cores. There remains full flexibility on the host or VM to bind kernel drivers or DPDK userspace poll mode drivers to each function, enabling both kernel and userspace-based networking applications.

Figure 4 provides a functional diagram showing the SmartNIC Represor pairing model.

Figure 4: Use Case: Interface Pairing with the SmartNIC Representer Pairing Model

2.3 Pairing Model Packet Flow

For a packet to be transmitted from one endpoint of a pair to a partner endpoint, the packet must traverse the internal loopback of the NIC. Since many classified flows in the transmit path may select the same internal endpoint for the destination of the flows, it is desirable to encode the destination endpoint on the packet before it traverses the loopback interface. This enables the receive path to only have a single classification entry per endpoint, and not have to replicate transmit classifications a second time after the loopback on the receive path.

The current pairing implementation adds a tunnel header encapsulation (most likely VXLAN) to the packet before it is transmitted to the loopback. The tunnel header contains information that encodes the destination pair endpoint. The receive path will classify the packet, de-encapsulate the tunnel header, and forward the packet to the specified pair endpoint.

The following diagram shows a typical SmartNIC application packet flow. In this application, all traffic destined to the x86 host and transmitted from the x86 host passes through the eight A72 Maia cores within Stingray. Interface pairs and Representer pairs are used to direct traffic between the x86 and A72s. Nitro's internal loopback capability enables this pairing behavior.

The red arrow shows data transmitted by the x86, via the SoC, to the wire. The blue arrow shows data received by the x86 from the wire via the SoC. Every packet passes through its original direction (RX or TX) twice, and the opposite direction once. The Nitro TX and RX pipelines in Stingray support a maximum of 45 MPPS. As a result, the PPS available for this application is $45/3 = 15$ MPPS.

2.4 SmartNIC Software Infrastructure Implementation

Components of the implementation include:

- User space configuration commands:
 - User space command to configure SmartNIC Interface and representor pairs.
 - User space command to configure tunnel.
 - User space command to configure in service software upgrade ISSU for SmartNIC representor and tunnel.
- DPDK support for SmartNIC representors including:
 - DPDK API for managing SmartNIC representors.
 - DPDK data path support for SmartNIC representors.
- HWRM and associated NIC firmware support for:
 - Configuring SmartNIC Interface Pairs and representors.
 - Configuring tunnels.
 - Modify existing representors and tunnel configuration to support ISSU.

2.5 User Space Configuration Commands

Bnxt-ctl is a user space command line utility to configure the new SmartNIC features as an extension of the existing bnxt-ctl application that is used to configure VF pairs for OVS offload. For SmartNIC on Stingray, bnxt-ctl is released as part of the southbound side rootfs and is expected to run from the southbound side (A72 CPUs) even though it could be compiled and run from the northbound side (host CPUs).

The bnxt-ctl application uses a Broadcom network interface kernel driver as proxy to communicate with chimp firmware through HWRM Nitro APIs. It uses netlink messages to communicate with kernel driver¹.

To help user scripting, the bnxt-ctl application is stateless and returns non-zero status if it runs into errors such as an error response from Chimp firmware.

To reduce running overhead such as time to invoke shell, bnxt-ctl application supports a batching mode that allows a user to execute up to ten commands in one batch.

2.6 Geographical Numbering of Hosts, Physical Functions, and Virtual Functions

A numbering scheme is required to unambiguously refer to a specific PF or VF on a Host, including the SoC. The geographical numbering logically identifies the hosts as follows:

- 0 – A72 host
- 1 to 4 – x86 hosts 1 to 4

Physical functions are indexed globally with the first PF on host 0 as 0. PF index on other hosts starts at total number of PFs on all hosts with host number smaller than that host. For example, assuming host 0 has 8 PFs and host 1 has 3 PFs, host 0 PFs will be indexed as 0 to 7, and host 1 PFs is indexed as 8 to 10. Virtual functions are indexed logically, relative to the specified physical function, starting at index 0².

1. Bnxt-ctl has a compile time option to use VFIO instead of netlink to send HWRM Nitro APIs to Chimp firmware. As of GA1 release, VFIO option has not been tested.
2. Bnxt-ctl design eventually changes so that physical functions are also indexed logically, relative to the specified host, starting at index 0. The change will be backward compatible with current design.

```
bnxt-ctl add-vf2fn-pair enP8p1s0f0 myPair vf 3 host 1 pf 8
```

The following command line is an example that is executed on the SoC to bind the fourth VF on a Broadcom Ethernet interface enP8p1s0f0 of the SoC with the fifth VF on the second PF on the first x86 host assuming there are eight PFs on the A72 host:

```
bnxt-ctl add-vf2fn-pair enP8p1s0f0 myPair vf 3 host 1 pf 8 vf 4
```

2.7 Create SmartNIC Represor Pairs

The following command is required to pair a represor on the SoC with a PF or VF on the x86 host:

```
bnxt-ctl add-rep2fn-pair <interface> [name] [host <index>] [pf <index>] [vf <index>]
```

The add-rep2fn-pair command pairs a represor on the local host with a PF or VF on any host (including the local host). The command takes the following parameters:

- <interface> – The name of the PF interface that is the parent of the local VF interface to be paired. When the Linux kernel driver is bound to the PF, this may be the Linux name of the Ethernet interface associated with the PF (for example, ethX). Alternatively, this name may be specified using the PCIe<domain>:<bus>:<slot>.<function> string of the PF (for example, 0000:05:00.1). The interface must be a Broadcom Ethernet interface owned by Linux even if PCIe naming is used.
- [name] – An optional name of the SmartNIC represor pair. This is used subsequently to reference the pair in other commands.
- host <index> – The logical index of the host containing the partner interface
- pf <index> – The global index of the PF on the host that is the partner PF interface, or the parent of the partner VF interface.
- [vf <index>] – The optional logical index of a VF that is the partner VF interface. If this option is omitted, the partner is a PF interface.

The following is a command line example that would be executed on the SoC to bind a named represor on a Broadcom Ethernet interface enP8p1s0f0 of the SoC with the first PF on the first x86 host assuming there are eight PFs on A72 host:

```
bnxt-ctl add-rep2fn-pair enP8p1s0f0 rep0 host 1 pf 8
```

The following is a command line example that would be executed on the SoC to bind a named Represor on a Broadcom Ethernet interface enP8p1s0f0 of the SoC with the fifth VF on the first PF on the first x86 host assuming there are eight PFs on A72:

```
bnxt-ctl add-rep2fn-pair enP8p1s0f0 rep0 host 1 pf 8 vf 4
```

2.8 Create SmartNIC PF Pairs

The following command is used to pair PF functions with another PF across multiple hosts:

```
bnxt-ctl add-pf-pair <interface> [name] host <index> pf <index>
```

The add-pf-pair command pairs a PF on the local host with a PF on any host (including the local host). The command takes the following parameters:

- <interface> – The name of the local PF interface to be paired. When the Linux kernel driver is bound to the PF, this may be the Linux name of the Ethernet interface associated with the PF (for example, ethX). Alternatively, this name may be specified using the PCIe<domain>:<bus>:<slot>.<function> string of the PF (for example, 0000:05:00.1). The interface must be a Broadcom Ethernet interface owned by Linux even if PCIe naming is used.
- [name] – A name of the PF pair. This is used subsequently to reference the pair in other commands.
- host <index> – The logical index of the host containing the partner interface.
- pf <index> – The global index of the PF on the host that is the partner PF interface, or the parent of the partner VF interface.

The following is a command line example that would be executed on the SoC to bind a named representor on a Broadcom Ethernet interface enP8p1s0f0 of the SoC with the first PF on the first x86 host assuming there are eight PFs on A72:

```
bnxt-ctl add-pf-pair enP8p1s0f0 pfpair0 host 1 pf 8
```

2.9 Pair Delete

The following command is used to delete interface pairs, representor pairs, or PF pairs:

```
bnxt-ctl del-pair <interface> <name>
```

The command takes following parameters:

- <interface> – The name of a Broadcom Ethernet interface. The interface is only used by bnxt-ctl to communicate with Chimp firmware. It does not have to be one of paired interface.
- <name> – Pair name used for pair creation.

2.10 Pair Query

The following command is used to get information and statistics of interface pairs, representor pairs or PF pairs:

```
bnxt-ctl show-pair <interface> <name>
```

The command takes following parameters:

- <interface> – The optional name of a Broadcom Ethernet interface. If the interface is not specified, bnxt-ctl goes through all valid Broadcom Ethernet interfaces owned by Linux and display all pairs on those interfaces. Pairs are not displayed if the endpoint interface is no longer owned by Linux, for example, transferred to DPDK.
- <name> – Optional parameter that is valid only when <interface> is specified. Specify pair name used for pair creation. If no name is specified, display all pairs with the specified interface as one end point.

2.11 DPDK Representor Enhancement For Pairing

A single device may be used to represent many partner endpoints. Each representor has associated RX and TX metadata which will be contained in received or transmitted mbufs for the purpose of demuxing and muxing the multiple Representors on the single port.

2.12 DPDK Datapath Support for SmartNIC Representors

The DPDK datapath support for SmartNIC Representors is straightforward. It uses the `udata64` field of the mbuf to support multiplexing/de-multiplexing of Representor traffic.

To transmit a frame to a host PCIe function, the DPDK application stores the TX handle of the SmartNIC Representor in the `udata64` field of the mbuf prior to initiating transmission of the frame on the locally-owned PF. If the frame is not destined for a host PCIe function, the `udata` fields must be zero. The TX handle is returned in the `tx_rep_id` parameter of the `rte_eth_dev_rep_get()` API function.

When a frame is received on the locally-owned PF, the DPDK application retrieves the RX handle of the SmartNIC representor from the `udata64` field of the mbuf. The RX handle identifies the host PCIe function that sourced the frame. The RX handle is returned in the `rx_rep_id` parameter of the `rte_eth_dev_rep_get` API function.

The `rte_eth_dev_rep_get` API is a single function call to query a named SmartNIC Representor previously created with `bnxt-ctl`.

```
int rte_eth_dev_rep_get(uint8_t port_id,
                        const char *repname,
                        uint32_t *rx_rep_id,
                        uint32_t *tx_rep_id)
```

Parameters:

- `port_id` – The port identifier of the Ethernet device.
- `repname` – A device specific name of the represented endpoint.
- `rx_rep_id` – A unique RX ID of the representor to be returned, if successful. It will be contained in the `udata64` field of the mbuf to identify packets received from the represented endpoint.
- `tx_rep_id` – A unique TX ID of the representor to be returned, if successful. It is stored in the `udata64` field of the mbuf to direct transmitted packets to the represented endpoint.

Returns:

- (0) if successful.
- (-ENOTSUP) if hardware does not support this feature.
- (-ENODEV) if `port_id` is invalid.
- (-EINVAL) if bad parameter.

The `filter_type` of `RTE_ETH_FILTER_TUNNEL` is used to match VXLAN packets. The operations `RTE_ETH_FILTER_ADD` and `RTE_ETH_FILTER_DELETE` will be used for the `filter_op` parameter. For the tunnel filter, `arg` is a pointer to a structure of type `struct rte_eth_tunnel_filter_conf`.

2.13 Enable OVS Forwarding

The following code fragment is used by the DPDK application to enable the OVS forwarding behavior:

```
int ret;
struct rte_eth_tunnel_filter_conf filter = {
    .tunnel_type = RTE_TUNNEL_TYPE_VXLAN,
};
ret = rte_eth_dev_filter_ctrl( port,
                              RTE_ETH_FILTER_TUNNEL,
                              RTE_ETH_FILTER_ADD,
                              &filter);
```

2.14 Disable OVS Forwarding

The following code fragment would be used by the DPDK application to disable the OVS forwarding behavior:

```
int ret;
struct rte_eth_tunnel_filter_conf filter = {
    .tunnel_type = RTE_TUNNEL_TYPE_VXLAN,
};
ret = rte_eth_dev_filter_ctrl( port,
                              RTE_ETH_FILTER_TUNNEL,
                              RTE_ETH_FILTER_DELETE,
                              &filter);
```

The DPDK `rte_eth_dev_filter_ctrl` API enables configuration of extensive hardware filtering functionality. This API is currently not implemented in the `bnxt` DPDK poll mode driver. The intent is to implement only enough functionality to support the specific cases highlighted in the code fragments above. This implementation requires that the Nitro firmware be sent HWRM messages to allocate or free an L2 filter to match VXLAN encapsulated packets from the port and direct those to the PF associated with the DPDK `eth_dev` object.

2.15 Standard and Custom Tunnels

Some SmartNIC applications need the ability to redirect packets of a specified tunnel type arriving on a port to a designated PF or VF for software processing.

The Nitro parser implementation natively supports parsing of standard tunnel encapsulations, including VXLAN, Geneve, L2GRE, as well as several others. The parser silicon also supports sets of parser registers for flexibly configuring additional non-native tunnel encapsulations. These additional encapsulations are handled by the User Parsed (UPAR) hardware. In addition, CFA classification is able to match on tunnel type as part of its L2 context lookup. This enables tunnel-type-specific features in silicon.

One tunnel encapsulation example is IPV4oVXLAN. It has an outer L2 header (with optional VLAN tags) followed by an IP header, followed by a UDP header, followed by a VXLAN header, followed by an inner IPv4 packet. Two attributes distinguish this encapsulation from a standard VXLAN tunnel. The destination UDP port in the UDP header does not use the standard destination port for VXLAN. The inner packet of the tunnel is IPv4, unlike standard VXLAN which carries an L2 packet as the inner packet.

IPv4oVXLAN encapsulation format example:

- Ethernet Header (IPv4oVXLAN with optional VLAN Tags)
- Outer IP Header:
 - Protocol = 0x11 (UDP)
- Outer UDP Header (this is the UPAR match criteria):
 - Destination Port = 4790 (IPv4oVXLAN)
- VXLAN Header:
 - The 24-bit VNI identifies the tunnel
 - Size of VXLAN header is 8B
- Inner Packet:
 - Type is IPv4

2.16 Bnxt-ctl Commands for Tunnels

The user space command to enable a tunnel and tunnel redirection is an extension of the existing `bnxt-ctl` command line tool. The following is a summary of the command set of `bnxt-ctl` after adding support for the new tunnel functionality:

- `config-tunnel`
- `add-tunnel-redirect`
- `del-tunnel-redirect`
- `show-tunnel-redirect`

2.17 Enabling and Disabling of Custom Tunnels

The following command is required to enable, disable, and configure custom tunnels:

```
bnxt-ctl cfg-tunnel control <ctrl-intf> <tunnel-type> dst_port [value]
```

When the command is issued with `dst_port` option and a value, the value is configured. When the command is issued with `dst_port` and no value, the currently configured value is removed. When custom tunnel is deleted, `bnxt-ctl` can potentially issue multiple HWRM APIs until the custom tunnel is removed from UPAR hardware configuration.

`Bnxt-ctl` application returns an error if the user tries to create duplicate IPv4oVXLAN custom tunnel or delete a non-existing IPv4oVXLAN custom tunnel.

The command takes the following parameters:

- `<interface>` – The name of a PF interface for sending the configuration. Tunnel configuration is global to a device. This is the Linux name of the Ethernet interface associated with the PF (for example, `ethX`).
- `<tunnel-type>` – Support `vlan_ipv4` only.
- `[value]` – Optional parameter for IPV4oVXLAN custom tunnel only. Specifies the destination port associated with the tunnel type. If not specified, IPV4oVXLAN tunnel is deleted.

The following is an example to enable IPV4oVXLAN tunnel:

```
bnxt-ctl cfg-tunnel eth0 vxlan_ipv4 dst_port 4790
```

The following is an example to disable IPV4oVXLAN tunnel:

```
bnxt-ctl cfg-tunnel eth0 vxlan_ipv4 dst_port
```

2.18 Configuring Tunnel Type Redirection

The following command is used to add and remove tunnel type redirection:

```
bnxt-ctl add-tunnel-redirect <interface> <tunnel-type>
bnxt-ctl del-tunnel-redirect <interface> <tunnel-type>
```

The command is issued on a PF interface and configures all packets of a specified tunnel type received on the associated port to be redirected to the PF or to the designated child VF. When multiple PFs are sharing the same network port, all packets destined to those PFs are redirected to the specified PF interface. For example, a PF on the northbound host side and a PF on the southbound SoC side are both mapped to network port 0, by adding an IPv4oVXLAN tunnel redirect to the southbound side SoC PF, all IPv4oVXLAN packets on port 0 will be redirected to the southbound PF on SoC.

Bnxt-ctl reports an error if the user tries to add a duplicate tunnel redirect for a port, or tries to delete a non-existing tunnel redirect on a port. If the user needs to change the tunnel redirect destination PF, `modify-tunnel-redirect` (see [In Service Software \(Hot\) Upgrade](#)) can be used.

The command takes the following parameters:

- `<interface>` – The name of a PF interface associated with the receive port. This is the Linux name of the Ethernet interface associated with the PF (for example, `ethX`).
- `<tunnel-type>` – `vlan_ipv4` or `vlan`.

The following command adds a tunnel redirect for all IPV4oVXLAN packets arriving on source port of `eth0`:

```
bnxt-ctl add-tunnel-redirect eth0 vlan_ipv4
```

The following command deletes the IPV4oVXLAN tunnel redirect on source port of `eth0`:

```
bnxt-ctl del-tunnel-redirect eth0 vlan_ipv4
```

2.19 Custom Tunnel UPAR Overview and Constraints

The number of custom tunnels that can be configured at any given time is limited by the number of hardware UPARs (two for Stingray).

The UPAR hardware configuration includes:

- A match criteria.
- A fixed tunnel header size.
- Offset/size/mask for extracting the tunnel ID and tunnel context from the tunnel header (these fields are available for use in some of the CFA lookup key formats).
- Specification of the inner packet type that follows the tunnel header.

As discussed in subsequent sections, the APIs support dynamic user configuration of the match criteria values. (for example, the UDP Dest Port that identifies IPv4oVXLAN could be dynamically configured to a value different than 4790). Unlike native tunnel types, custom tunnels are not enabled at firmware initialization time. As a result, a custom tunnel is only enabled after the user or application configures the match criteria for the custom tunnel.

The characteristics of the UPAR hardware impose a number of constraints on the operation of custom tunnels. One important constraint is that all frames that match the criteria for a given custom tunnel configuration must have the same format. This means that they must have the same tunnel header size and inner packet type. If these conditions are not met, incorrect parsing may occur. In some cases, incorrect parsing may result in packet corruption (for example, TCP checksum offload on transmit).

To keep things simple, support is limited to configuration of a single instance of each custom tunnel encapsulation. For example, this means that only a single match criterion (UDP destination port) may be configured for the IPv4oVXLAN tunnel encapsulation.

2.20 In Service Software (Hot) Upgrade³

A primary SmartNIC use case is the execution of a software virtual switch application (typically DPDK based), running on the embedded A72 CPU cores of the Stingray. The application switches packets between the physical ports and PCIe virtual functions on the x86 host system.

To maintain high availability of the system, perform service upgrades of the virtual switch application. There are many aspects of ISSU. This implementation of the SmartNIC packet steering infrastructure enables ISSU for the virtual switch application with minimal or even zero packet loss. The responsibility of the user application regarding ISSU includes their own control state synchronization, management of the CPU resources during the ISSU, acceptance of newer version and potential rollback to existing version, or other such scenarios.

The flows of traffic received on each PF from the Ethernet ports as well as the flows of traffic received on the representor PF from the partner VFs are all asynchronous to each other. As a result, the switchover from an existing version to a newer version can be performed by independently reconfiguring each flow to be directed to the newer version PFs instead of the existing version PFs.

Two new `bnxt-ctl` commands are introduced:

- `bnxt-ctl modify-tunnel-redirect` – Performs reconfiguration of the PF associated with an existing tunnel redirection.
- `bnxt-ctl modify-rep2fn-pair` – Performs reconfiguration of the PF associated with an existing SmartNIC representor pair.

An important characteristic of these commands is that the reconfiguration utilizes the currently configured CFA resources and therefore cannot fail due to inability to allocate new resources. A second important characteristic is that each of these commands performs an atomic operation to the CFA, enabling the reconfiguration to occur while traffic is flowing. There is zero packet loss due to reconfiguration operations. This implies that it is possible to perform an in service software upgrade of the User application with no disruption to traffic. While this is true, in practice traffic disruption during ISSU may occur for other reasons. One potential cause of traffic disruption may be the inability for two running instances of the User Application to effectively share the CPU resources in order to keep up with the traffic load. Generally, it is good practice to schedule ISSU operations to be performed during maintenance windows when there are reduced traffic loads. These issues are within the domains of the user application and the operational process of performing software upgrades.

3. Current ISSU design and implementation are for representor pairing model only, support for the other pairing model is TBD and outside the scope of this document.

2.21 ISSU Infrastructure Implementation

Components of the implementation include:

- User space configuration commands
 - User space command to modify the configuration of tunnel redirection for switchover from existing version to a new version.
 - User space command to modify the configuration SmartNIC representor pairs for switchover from existing version to a new version.
- DPDK support for SmartNIC representor ISSU:
 - DPDK API for querying SmartNIC representors new version must be able to query representors currently in use by existing version.
- NIC firmware support for:
 - Modifying the configuration of tunnel redirection.
 - Modifying the configuration of SmartNIC representor pairs.

2.22 ISSU User Space Configuration Commands

The user space command to reconfigure tunnel redirection and SmartNIC representor pairs are sub-commands of the `bnxt-ctl` command. The following two commands are added:

- `bnxt-ctl modify-tunnel-redirect`
- `bnxt-ctl modify-rep2fn-pair`

The modify version of the commands will have similar syntax as the add version of the commands that are described in the specifications. The modify commands operate on PF interfaces that may be currently in use by DPDK. As a result, the PF interface itself cannot be used by the `bnxt-ctl` command as a control channel. The modify commands allow for a `ctrl-intf` parameter to be specified, allowing a different interface to be used as the control channel for the command.

To minimize overhead and total execution time, `bnxt-ctl` supports a batch execution mode. The batch execution allows up to 10 commands to be issued in one `bnxt-ctl` line. `Bnxt-ctl` is used as the keyword to separate individual commands. If any command in the batch failed, the remaining commands are not executed.

The `modify-tunnel-redirect` command is issued to modify the PF or VF index of an existing tunnel redirection. The new PF must be mapped to the same network port as current destination PF of the tunnel. The interface parameter specifies a PF that is associated with a port currently redirecting packets of the specified tunnel-type. If no VF parameter is specified, the PF specified in the interface parameter will be the new destination of the tunnel packets. If the VF parameter is present, the index parameter is used to specify the VF to be used as the new destination of the tunnel packets.⁴

The command takes the following parameters:

- `<interface>` – Must be of PCI naming and not owned by Linux as representors are not dynamically generated. Modify is only valid after ownership is given to, for example, DPDK.
- `name` – Must be a name of a known pair, but verification is first done in the firmware.
- `ctrl-intf` – Mandatory and must be a valid Linux interface, no PCI naming.
- `all` – Switch all representor pairs sharing same PF as the end point.

4. The optional VF parameter is not supported for Stingray.

2.23 User Space Configuration Commands Examples

This command modifies an existing IPV4oVXLAN tunnel redirect to use PF 0008:01:00.0 as new destination PF of all IPV4oVXLAN packets using a control interface enP8p1s0f7d1. The <interface> parameter could be a PCI name such as 0008:01:00.0 as in the example or a valid and link up Broadcom Linux network interface. If <interface> is a Broadcom Linux network interface, the <ctrl-intf> parameter is optional and <ctrl-intf> is assumed to be same as <interface>. If <interface> is PCI name, the <ctrl-intf> parameter enP8p1s0f7d1 is mandatory and must be a valid and link up Broadcom Linux network interface.

```
bnxt-ctl modify-tunnel-redirect 0008:01:00.0 vxlan_ipv4 control enP8p1s0f7d1
```

This command modifies an existing representor pair named as beitest4 to use PF 0008:01:00.5 as a new end point using a control interface enP8p1s0f7d1. The <interface> parameter 0008:01:00.5 must follow PCI naming and not be owned by Linux. The modify-rep2fn-pair command is only valid after the ownership of the PF 0008:01:00.5 is given to, for example, DPDK. The <ctrl-intf> parameter enP8p1s0f7d1 must be a valid and a link up Broadcom Linux network interface.

```
bnxt-ctl modify-rep2fn-pair 0008:01:00.5 beitest4 control enP8p1s0f7d1
```

Following command is similar to the above modify-rep2fn-pair command other than the all option, it modifies all existing representor pairs that share same PF endpoint as representor pair beitest4 to use PF 0008:01:00.5 as the new endpoint. For example, assuming beitest4 is one of 128 representor pairs created, and all representor pairs have PF 0008:01:00.4 as an endpoint, the following command modifies all 128 representor pairs to use PF 0008:01:00.5 as the new endpoint.

```
bnxt-ctl modify-rep2fn-pair 0008:01:00.5 beitest4 control enP8p1s0f7d1 all
```

This command batches two commands, modify-tunnel-redirect and modify-rep2fn-pair, in one command line.

```
bnxt-ctl modify-tunnel-redirect 0008:01:00.0 vxlan_ipv4 control enP8p1s0f7d1 bnxt-ctl modify-rep2fn-pair 0008:01:00.5 beitest4 control enP8p1s0f7d1 all
```

Appendix A: Acronyms and Abbreviations

Table 2 lists the acronyms and abbreviations used in this document.

For a more complete list of acronyms and other terms used in Broadcom documents, go to: <http://www.broadcom.com/press/glossary.php>.

Table 2: Acronyms and Abbreviations

Item	Comment
AP	Application Processor, also referred to as Maia
BITW	Bump In The Wire
Bono	RDMA/RoCE control firmware running on a Cortex-R5 inside Nitro-SR
CFA	Configurable Flow Accelerator
ChiMP	L2/PF/VF control firmware running on a Cortex-M3 inside Nitro-SR, implements the HWRM API
Cortex-A72	High speed Cortex-A processors from ARM. These cores on Stingray make up main CPUs on the southbound side
CQ	RDMA Completion Queue
Cumulus	High speed network controller, also known as Nitro
DPDK	Data Plane Development Kit
EP	PCIe endpoint
FlexSparx4	Stingray block that contains flow accelerator engines (that is, PAE, compression, encryption, and so on)
FMR	Fast Physical Memory Region
FR-PMR	Fast Register Physical Memory Region
Host	Northbound server host connected via PCIe, Stingray is seen as a PCIe endpoint to the host
HSI	Hardware/Software Interface
HWRM	Hardware Resource Manager (implemented in ChiMP)
ISSU	In Service Software Upgrade
L2oQP	L2 packet interface used to pass packets between the northbound and southbound sides of Stingray
LB	Load Balancing
LDK	Broadcom's Linux Distribution for iProc-based SoCs
Maia	Former codename for Cortex-A72. Marketing/Engineering term for the Cortex-A72
MHB	PCIe Multiple Host Bridge
MR	RDMA Memory Region
MW	RDMA Memory Window bound to portion of an MR
NIC	Network Interface Card
Nitro	Nitro high speed network controller
Northbound	Host side outside of Stingray.
OFED	OpenFabrics Enterprise Distribution
OoBM	Out Of Band Management
OVS	Open vSwitch, an open-source implementation of a distributed virtual multi-layer switch.
PD	RDMA Protection Domain
PF	Physical Function
PMD	Poll Mode Driver, a user-mode driver that completely bypasses the kernel
PMR	Physical Memory Region
QP	RDMA Queue Pair

Table 2: Acronyms and Abbreviations (Continued)

Item	Comment
QPLib	Common driver code for using the RoCE/RoPE engine in Nitro-SR
RC	PCIe root-complex
RDMA	Remote Direct Memory Access
RNIC	RDMA NIC
RoCE	RDMA over Converged Ethernet
RoPE	RDMA over PCIe Engine
RQ	RDMA Receive Queue
RSS	Receive Side Scaling
SDP	Software Development Platform
SERoQP	Serial interface used to pass data between the northbound and southbound sides of Stingray (management console)
SmartNIC	Inline traffic flow processing by the Maia cores
SoC	System on a Chip
Southbound	ARM SoC side inside Stingray.
SQ	RDMA Send Queue
SR-IOV	PCIe Single Root I/O Virtualization
SRQ	RDMA Shared Receive Queue
UPAR	User Parsed Register
VF	Virtual Function

Revision History

5880X-UG302; January 31, 2020

Updated:

- BCM5880X Hardware Platforms

5880X-UG301; November 1, 2018

Removed:

- Removed SmartNIC figure.

Updated:

- BCM5880X Hardware Platforms

5880X-UG300; August 15, 2018

Initial Release.

